

HiDiTingV100 音频驱动

开发指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档主要介绍音频驱动开发相关内容，包括工作原理、按场景描述接口使用方法和注意事项，用于指导客户快速熟悉音频接口。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 概述.....1

2 SYS.....2

2.1 概述..... 2

2.2 开发指引..... 2

2.2.1 接口说明..... 2

3 ADP4

3.1 概述..... 4

3.2 开发指引..... 4

3.2.1 接口说明..... 4

3.2.2 数据结构..... 5

4 SOUND.....7

4.1 概述..... 7

4.2 开发指引..... 9

4.2.1 接口说明..... 9

4.2.2 音频播放场景 10

5 ADEC.....14

5.1 概述..... 14

5.2 开发指引..... 14

5.2.1 接口说明..... 14

5.2.2 解码播放场景 15

5.3 注意事项..... 19

6 AENC.....20

6.1 概述..... 20

6.2 开发指引.....	20
6.2.1 接口说明.....	20
6.2.2 用户送帧场景	21
6.3 注意事项.....	25
7 AI	26
7.1 概述.....	26
7.2 开发指引.....	27
7.2.1 接口说明.....	27
7.2.2 音频录制场景	30
8 SEA	34
8.1 概述.....	34
8.2 开发指引.....	35
8.2.1 接口说明.....	35
8.2.2 数据结构.....	40
8.2.3 语音识别场景	41
8.2.4 语音通话场景	44
8.3 注意事项.....	48
9 VAD	49
9.1 概述.....	49
9.2 开发指引.....	49
9.2.1 接口说明.....	49
9.2.2 语音活动检测场景	50
9.3 注意事项.....	51
10 VENDOR	52
10.1 概述.....	52
10.2 开发指引.....	52
10.2.1 接口说明.....	52
11 ANC	53
12 HAID	54

插图目录

图 4-1 SOUND 设备关系图..... 7

图 4-2 SOUND 设备框图 8

图 7-1 AI 的数据流处理过程..... 26

表格目录

表 7-1 AI 属性参数说明 29

表 8-1 SEA 能力集合配置项 36

表 8-2 SEA 实例属性 37

表 8-3 SEA 语音增强引擎属性..... 37

表 8-4 SEA 语音 AI 引擎属性..... 38

1 概述

本文档主要介绍音频驱动开发相关内容，重点介绍了音频驱动各个模块以及各个模块的开发指引。

2 SYS

2.1 概述

2.2 开发指引

2.1 概述

SYS (System) 为音频系统全局管理模块。

2.2 开发指引

2.2.1 接口说明

SYS 模块实现了以下功能：

- 音频系统初始化、去初始化功能，该功能模块提供以下 API：
 - uapi_audio_init：初始化音频系统。
 - uapi_audio_deinit：去初始化音频系统。
- TWS 耳机管理功能，该功能模块提供以下 API：
 - uapi_audio_set_product_form：设置音频产品形态。(该接口暂不支持)
 - uapi_audio_set_tws_mono_mode：设置真无线耳机单耳模式。(该接口暂不支持)
 - uapi_audio_set_tws_mode：设置真无线耳机工作模式。(该接口暂不支持)
 - uapi_audio_set_tws_role：设置真无线耳机角色。(该接口暂不支持)
 - uapi_audio_set_dsp_log_level：设置 DSP 打印等级。

- 音频系统调试功能，该功能模块提供以下 API：
 - uapi_audio_set_debug_cfg：设置调试功能配置。
 - uapi_audio_get_debug_cfg：获取调试功能配置。

Draft

3 ADP

3.1 概述

3.2 开发指引

3.1 概述

ADP (Audio Data Port) 即音频数据的输入输出模块。

- ADP 作为输入时, 可以接收外部的数据, 将数据输送给后一级模块。
- ADP 作为输出时, 可以通过接口向外提供音频数据。

📖 说明

一个 ADP 实例只能有一种工作模式, 即单独作为输入模块或单独作为输出模块。

同时最多允许打开 8 个 ADP 实例。

3.2 开发指引

3.2.1 接口说明

ADP 模块实现了以下功能:

- ADP 模块初始化、去初始化功能, 该功能模块提供以下 API:
 - uapi_adp_init: 初始化 ADP 模块。
 - uapi_adp_deinit: 去初始化 ADP 模块。
- ADP 实例管理, 用于创建和销毁 ADP 实例, 该功能模块提供以下 API:
 - uapi_adp_get_def_attr: 获取 ADP 默认属性。

- uapi_adp_create: 创建一路 ADP。
- uapi_adp_destroy: 销毁一路 ADP。
- uapi_adp_map: 将 ADP 映射到当前进程空间, 或者将其他业务核心创建的 ADP 映射到当前业务核心。
- uapi_adp_unmap: 解除 ADP 在当前进程空间或业务核心的映射。
- uapi_adp_get_attr: 获取 ADP 属性。
- uapi_adp_set_attr: 设置 ADP 属性。
- ADP 与其他模块级联功能, 该功能模块提供以下 API:
 - uapi_adp_attach_output: 为 ADP 绑定一个输出。
 - uapi_adp_detach_output: 解除 ADP 与输出的绑定关系。
- ADP 事件上报功能, 该功能模块提供以下 API:

uapi_adp_register_event_proc: 注册 ADP 事件处理回调函数, 事件包括: 码流帧信息变化、获取 EOS、上报 EOS、上报 PTS、获取延时。
- ADP 数据输入输出功能, 该功能模块提供以下 API:
 - uapi_adp_query_free: 查询 ADP 中剩余空间大小。
 - uapi_adp_query_busy: 查询 ADP 中已使用空间大小。
 - uapi_adp_get_buffer: 获取一块 ADP buffer。
 - uapi_adp_put_buffer: 写完数据后, 更新 ADP buffer。
 - uapi_adp_send_stream: 向 ADP buffer 送入音频 ES 流。
 - uapi_adp_acquire_stream: 从 ADP buffer 获取音频 ES 流。
 - uapi_adp_release_stream: 释放 ADP buffer 中的音频 ES 流。
 - uapi_adp_send_frame: 向 ADP buffer 送入音频 PCM 数据。
 - uapi_adp_acquire_frame: 从 ADP buffer 获取音频 PCM 数据。
 - uapi_adp_release_frame: 释放 ADP buffer 中的 PCM 数据。

3.2.2 数据结构

以下为 ADP 模块部分数据结构的参数说明和范围限制:

说明

SYS_SHM_MAX_SIZE = 0x10000 - 0x1000 = 0xF000, 为 DSP 共享内存的大小。

- uapi_adp_attr
 - buf_size: ADP 缓冲区大小。

参数范围: [0, SYS_SHM_MAX_SIZE]

- uapi_stream_buf

- data: 数据缓冲区地址。
- size: 数据大小。

参数范围: [0, adp_create 时的 buf_size 大小-8]

- pts: 时间戳 (单位: μ s)。
- eos: 流结束标志。

参数范围: 0、1

- pkg_loss: 丢包标志。

参数范围: 0、1

- uapi_audio_frame

- interleaved: 数据是否交织。

参数范围: 0、1

- channels: 声道数。

参数范围: 1、2、4、6、8、16

- bit_depth: 位宽。

参数范围: 16、24、32

- sample_rate: 采样率。

参数范围: [4000, 768000]

- pts: 时间戳 (单位: μ s)。

- pcm_buffer: PCM 数据缓冲指针 (该参数未使用)。

- bits_buffer: 透传数据缓冲指针。

- bits_bytes: 透传数据长度。

参数范围: [0, adp_create 时的 buf_size 大小-8]

- pcm_samples: PCM 数据采样点数量。

- frame_index: 帧序号。

- eos: 流结束标志。

参数范围: 0、1

- pkg_loss: 丢包标志。

参数范围: 0、1

4 SOUND

4.1 概述

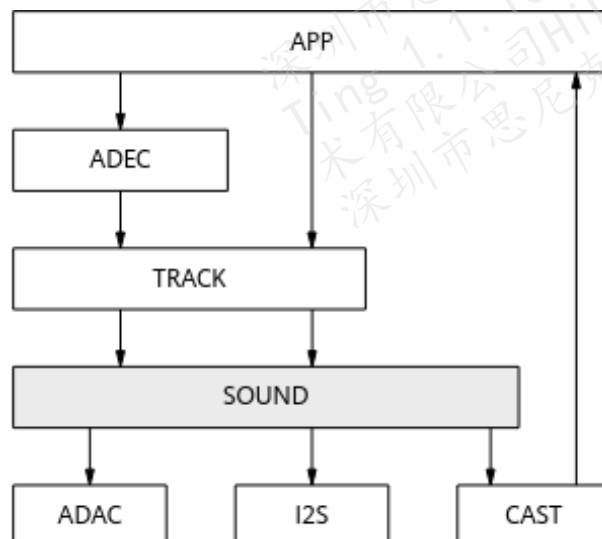
4.2 开发指引

4.1 概述

SOUND 是虚拟声卡，能将 PCM 数据输出到 ADAC、I2S 等物理端口及 CAST 虚拟端口，并支持多路同时输出，为用户带来丰富的音频应用。

SOUND 在系统中的位置如图 4-1 所示。

图4-1 SOUND 设备关系图

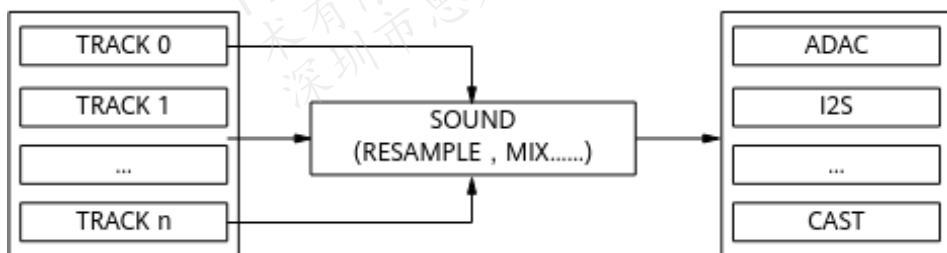


- SOUND：虚拟声卡，定义 3 个 SOUND 设备（根据需要可打开 0~3 个，且可任意组合）：

- UAPI_SND_0
- UAPI_SND_1
- UAPI_SND_2

SOUND 设备将各路 TRACK 输入的数据进行处理（包括重采样、混音等）后，输出到绑定的 OUTPORT，如图 4-2 所示。

图4-2 SOUND 设备框图



- TRACK：音频数据播放通路。多路 TRACK 的数据会在 SOUND 中进行混音后输出。
- OUTPORT：音频物理输出端口，包括 ADAC OUTPORT 和 I2S OUTPORT 等。支持对 OUTPORT 进行静音、音量、声道模式设置。
- CAST：通过使能音频 CAST 功能会虚拟出一个音频 CAST OUTPORT，该 OUTPORT 可投射出系统所有解码输出的音频数据。
- 声道模式：支持对单个 OUTPORT 或所有 OUTPORT 进行声道模式设置。
- 音量：支持两级音量设置，即 TRACK 音量设置和 OUTPORT 音量设置：
 - TRACK 音量支持对各路 TRACK 单独进行音量设置。
 - OUTPORT 音量设置支持对单个 OUTPORT 或所有 OUTPORT 进行音量设置。

TRACK 和 OUTPORT 只支持设置绝对音量，音量的设置范围是 -81dB ~ +18dB，步长为 0.125dB。

📖 说明

同时最多允许打开 2 个 SOUND 设备，4 路 TRACK 通路和 4 个 OUTPORT 端口。

4.2 开发指引

4.2.1 接口说明

SOUND 模块实现了以下功能：

- SOUND 模块初始化、去初始化功能，该功能模块提供以下 API：
 - uapi_snd_init：初始化 SOUND 模块。
 - uapi_snd_deinit：去初始化 SOUND 模块。
- SOUND 设备管理功能，实现 SOUND 设备打开、关闭等功能，该功能模块提供以下 API：
 - uapi_snd_get_default_attr：获取 SOUND 设备默认属性。
 - uapi_snd_open：打开某个 SOUND 设备。
 - uapi_snd_close：关闭某个 SOUND 设备。
- OUTPORT 管理功能，该功能模块提供以下 API：
 - uapi_snd_set_port_enable：设置 OUTPORT 的使能状态。
 - uapi_snd_get_port_enable：获取 OUTPORT 的使能状态。
 - uapi_snd_set_mute：设置 OUTPORT 的静音状态。
 - uapi_snd_get_mute：获取 OUTPORT 的静音状态。
 - uapi_snd_set_volume：设置 OUTPORT 音量。
 - uapi_snd_get_volume：获取 OUTPORT 的音量值。
 - uapi_snd_set_track_mode：设置 OUTPORT 声道模式。
 - uapi_snd_get_track_mode：获取 OUTPORT 的声道模式。
 - uapi_snd_attach_output：为指定的 OUTPORT 绑定一个输出。
 - uapi_snd_detach_output：解除 OUTPORT 与输出的绑定关系。
- 音效设置功能，该功能模块提供以下 API：
 - uapi_snd_get_aef_param：获取算法参数。
 - uapi_snd_set_aef_param：设置算法参数。
 - uapi_snd_set_aef_profile：设置 SOUND 的音效模式。
- TRACK 管理功能，该功能模块提供以下 API：
 - uapi_snd_get_track_default_attr：获取 TRACK 默认属性。
 - uapi_snd_create_track：创建一路 TRACK。

- `uapi_snd_destroy_track`: 销毁一路 TRACK。
- `uapi_snd_get_track_state`: 获取 TRACK 的运行状态。
- `uapi_snd_track_start`: 启动 TRACK, 开始播放。
- `uapi_snd_track_stop`: 停止 TRACK, 停止播放。
- `uapi_snd_track_pause`: 暂停 TRACK, 暂停播放。
- `uapi_snd_track_resume`: 启动 TRACK, 恢复播放。
- `uapi_snd_get_track_delay`: 获取 TRACK 的延时。
- `uapi_snd_get_track_play_pts`: 获取 TRACK 正在播放的音频的时间戳。
- `uapi_snd_set_track_mute`: 设置一路 TRACK 的静音状态。
- `uapi_snd_get_track_mute`: 获取一路 TRACK 的静音状态。
- `uapi_snd_set_all_track_mute`: 设置所有 TRACK 的静音状态。
- `uapi_snd_get_all_track_mute`: 获取所有 TRACK 的静音状态。
- `uapi_snd_set_track_volume`: 设置 TRACK 音量。
- `uapi_snd_get_track_volume`: 获取 TRACK 音量。
- `uapi_snd_attach_track_output`: 为 TRACK 绑定一路输出。
- `uapi_snd_detach_track_output`: 解除 TRACK 与输出的绑定关系。

说明

`uapi_snd_set_aef_profile`、`uapi_snd_set_all_track_mute` 和 `uapi_snd_get_all_track_mute` 接口暂不支持。

若使用了内置的 88262 音效算法, 调用 `uapi_snd_open` 时需将采样率设为 48kHz。

4.2.2 音频播放场景

音频播放场景是指将 PCM 数据送至 TRACK, 接着 SOUND 对各路 TRACK 的数据进行重采样、混音等处理后, 在 SOUND 所绑定的物理端口输出。输出过程中可以对 OUTPORT 进行音量设置、静音设置等操作。

说明

在调用其他 SOUND 接口之前, 必须先将 SOUND 模块初始化; 不再使用 SOUND 模块时, 须将 SOUND 模块去初始化。

音频播放初始化步骤

音频播放初始化步骤如下:

步骤 1 调用 uapi_adp_init 接口初始化 ADP 模块。

```
uapi_adp_init();
```

步骤 2 调用 uapi_snd_init 接口初始化 SOUND 模块。

```
uapi_snd_init();
```

步骤 3 调用 uapi_snd_get_default_attr 接口获取 SOUND 设备默认属性。

```
uapi_snd snd_id;  
uapi_snd_attr snd_attr;  
uapi_snd_get_default_attr(snd_id, &snd_attr);
```

步骤 4 修改属性，调用 uapi_snd_open 接口打开音频输出设备。

```
td_handle h_snd = 0;  
snd_attr.port_num = 1;  
snd_attr.port_attr[0].out_port = UAPI_SND_OUT_PORT_I2S0;  
snd_attr.channels = UAPI_AUDIO_CHANNEL_2;  
snd_attr.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;  
snd_attr.sample_rate = UAPI_AUDIO_SAMPLE_RATE_48K;  
uapi_snd_open(&h_snd, snd_id, &snd_attr);
```

步骤 5 调用 uapi_snd_get_track_default_attr 接口获取 TRACK 默认属性。

```
uapi_snd_track_attr track_attr;  
uapi_snd_get_track_default_attr(&track_attr);
```

步骤 6 修改属性，并调用 uapi_snd_create_track 接口创建 TRACK。

```
td_handle h_track = 0;  
uapi_snd_create_track(&h_track, h_snd, &track_attr);
```

步骤 7 调用 uapi_adp_get_def_attr 接口获取 ADP 的默认属性。

```
uapi_adp_attr adp_attr;  
uapi_adp_get_def_attr(&adp_attr);
```

步骤 8 调用 uapi_adp_create 接口创建一路 ADP。

```
td_handle h_adp = 0;  
uapi_adp_create(&h_adp, &adp_attr);
```

步骤 9 调用 uapi_adp_attach_output 接口将 ADP 的输出设置为 TRACK。

```
uapi_adp_attach_output(h_adp, h_track);
```

步骤 10 调用 uapi_snd_track_start 接口启动 TRACK。

```
uapi_snd_track_start(h_track, TD_NULL);
```

步骤 11 创建一个送音频帧线程，反复调用 `uapi_adp_send_frame` 接口向 ADP 送入 PCM 数据。若要实现多路音频播放，可以重复步骤 5 ~ 步骤 11。

```
td_s32 ret;
td_bool task_active = TD_TRUE;
uapi_audio_frame frame;
while (task_active) {
    read_frame(&frame); /* read_frame需用户自行开发，用于获取音频帧 */

    while (task_active) {
        ret = uapi_adp_send_frame(h_adp, &frame);
        if (ret != EXT_SUCCESS) {
            continue;
        }
        break;
    }
}
```

----结束

说明

TRACK 没有音频数据接口，需要创建一路 ADP，并且将 TRACK 绑定到 ADP 的输出。

音频播放去初始化步骤

音频播放去初始化步骤如下：

步骤 1 销毁发送音频帧线程。

步骤 2 调用 `uapi_snd_track_stop` 停止 TRACK。

```
uapi_snd_track_stop(h_track);
```

步骤 3 调用 `uapi_adp_detach_output` 接口解除 ADP 与 TRACK 的绑定关系。

```
uapi_adp_detach_output(h_adp, h_track);
```

步骤 4 调用 `uapi_adp_destroy` 接口销毁 ADP 通路。

```
uapi_adp_destroy(h_adp);
```

步骤 5 调用 `uapi_snd_destroy_track` 接口销毁 TRACK。如果创建了多路音频播放，则需要为每一路执行步骤 1 ~ 步骤 5。

```
uapi_snd_destroy_track(h_track);
```

步骤 6 调用 `uapi_snd_close` 接口关闭 SOUND。

```
uapi_snd_close(h_snd);
```

步骤 7 调用 uapi_snd_deinit 接口去初始化 SOUND 模块。

```
uapi_snd_deinit();
```

步骤 8 调用 uapi_adp_deinit 接口去初始化 ADP 模块。

```
uapi_adp_deinit();
```

----结束

Draft

5 ADEC

5.1 概述

5.2 开发指引

5.3 注意事项

5.1 概述

ADEC (Audio Decoder) 即音频解码模块。ADEC 是解码框架，对于系统支持的音频格式，ADEC 都可以完成解码工作。

ADEC 模块接收前级模块发送的码流数据，根据码流格式调用对应格式的解码库，解码出 PCM 数据发送到后端模块。例如，SOUND 模块。

说明

ADEC 模块支持 MP3、AAC、SBC 等音频类型。

sample_decode 不支持 FLAC。

同时最多允许打开 2 个 ADEC 实例。

5.2 开发指引

5.2.1 接口说明

ADEC 模块实现了以下功能：

- ADEC 模块系统管理功能，实现初始化、去初始化等功能，该功能模块提供以下 API：

- uapi_adec_init: 初始化 ADEC 模块。
- uapi_adec_deinit: 去初始化 ADEC 模块。
- ADEC 模块实例管理功能, 该功能模块提供以下 API:
 - uapi_adec_create: 创建一路音频解码实例。
 - uapi_adec_destroy: 销毁一路音频解码实例。
 - uapi_adec_get_attr: 获取音频解码实例的属性。
 - uapi_adec_set_attr: 设置音频解码实例的属性。
 - uapi_adec_start: 启动音频解码实例。
 - uapi_adec_stop: 停止音频解码实例。
- ADEC 与其他模块级联功能, 该功能模块提供以下 API:
 - uapi_adec_attach_output: 为音频解码实例绑定一个输出。
 - uapi_adec_detach_output: 为音频解码实例解绑一个输出。
- ADEC 事件上报功能, 该功能模块提供以下 API:
 - uapi_adec_register_event_proc: 注册 ADEC 事件处理回调函数。

解码器实例属性如下:

- codec_id: 每种音频格式都对应唯一 ID。
- buf_dur_ms: 解码输出帧大小对应的时间。
- param: 其他参数, 主要包括声道数、位宽、采样率、私有数据地址以及长度。

📖 说明

uapi_adec_register_event_proc 接口暂不支持。

5.2.2 解码播放场景

用户根据自己的实际需求, 将需要解码播放的码流通过 ADP 实例送到 ADEC 模块进行解码, 并通过 SOUND 模块播放。

📖 说明

在调用其他 ADEC 接口之前, 必须先对 ADEC 模块初始化; 不再使用 ADEC 模块时, 须将 ADEC 模块去初始化。

解码播放初始化步骤

解码播放初始化步骤如下:

步骤 1 调用 uapi_adp_init 接口初始化 ADP 模块。

```
uapi_adp_init();
```

步骤 2 调用 uapi_snd_init 接口初始化 SOUND 模块。

```
uapi_snd_init();
```

步骤 3 调用 uapi_adec_init 接口初始化 ADEC 模块。

```
uapi_adec_init();
```

步骤 4 调用 uapi_snd_get_default_attr 接口获取 SOUND 设备默认属性。

```
uapi_snd_snd_id;  
uapi_snd_attr_snd_attr;  
uapi_snd_get_default_attr(snd_id, &snd_attr);
```

步骤 5 修改属性，调用 uapi_snd_open 接口打开音频输出设备。

```
td_handle h_snd = 0;  
snd_attr.port_num = 1;  
snd_attr.port_attr[0].out_port = UAPI_SND_OUT_PORT_I2S0;  
snd_attr.channels = UAPI_AUDIO_CHANNEL_2;  
snd_attr.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;  
snd_attr.sample_rate = UAPI_AUDIO_SAMPLE_RATE_48K;  
uapi_snd_open(&h_snd, snd_id, &snd_attr);
```

步骤 6 调用 uapi_snd_get_track_default_attr 接口获取 TRACK 默认属性。

```
uapi_snd_track_attr track_attr;  
uapi_snd_get_track_default_attr(&track_attr);
```

步骤 7 修改属性，并调用 uapi_snd_create_track 接口创建 TRACK。

```
td_handle h_track = 0;  
uapi_snd_create_track(&h_track, h_snd, &track_attr);
```

步骤 8 调用 uapi_adec_create 接口创建一路 ADEC，用于音频帧的解码。

```
td_handle h_adec = 0;  
uapi_adec_create(&h_adec);
```

步骤 9 调用 uapi_adec_get_attr 接口获取 ADEC 的属性。

```
uapi_adec_attr adec_attr;  
uapi_adec_get_attr(h_adec, &adec_attr);
```

步骤 10 修改属性，指定使用 codec_id，调用 uapi_adec_set_attr 接口设置新属性。

```
adec_attr.codec_id = UAPI_ACODEC_ID_MSBC;  
adec_attr.param.sample_rate = UAPI_AUDIO_SAMPLE_RATE_16K;
```

```

adec_attr.param.channels = UAPI_AUDIO_CHANNEL_1;
adec_attr.param.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;
uapi_adec_set_attr(h_adec, &adec_attr);

```

步骤 11 调用 uapi_adec_attach_output 接口将 ADEC 的输出设置为 TRACK。

```
uapi_adec_attach_output(h_adec, h_track);
```

步骤 12 调用 uapi_adp_get_default_attr 接口获取输入 ADP 的默认属性。

```

uapi_adp_attr adp_attr;
uapi_adp_get_def_attr(&adp_attr);

```

步骤 13 调用 uapi_adp_create 接口创建一路输入 ADP，用于给 ADEC 送码流数据。

```

td_handle h_adp = 0;
uapi_adp_create(&h_adp, &adp_attr);

```

步骤 14 调用 uapi_adp_attach_output 接口将输入 ADP 的输出设置为 ADEC。

```
uapi_adp_attach_output(h_adp, h_adec);
```

步骤 15 调用 uapi_snd_track_start 启动 TRACK。

```
uapi_snd_track_start(h_track, TD_NULL);
```

步骤 16 调用 uapi_adec_start 接口启动 ADEC 进行编码。

```
uapi_adec_start(h_adec);
```

步骤 17 创建一个送音频帧线程，反复调用 uapi_adp_send_stream 向 ADP 送入码流数据。若
要实现多路音频解码，可以重复步骤 6 ~ 步骤 17。

```

td_s32 ret;
td_bool task_active = TD_TRUE;
uapi_stream_buf stream;
while (task_active) {
    read_stream(&stream); /* read_stream需用户自行开发，用于获取码流数据 */
    while (task_active) {
        ret = uapi_adp_send_stream(h_adp, &stream);
        if (ret != EXT_SUCCESS) {
            continue;
        }
        break;
    }
}

```

----结束

解码播放去初始化步骤

解码播放去初始化步骤如下：

步骤 1 销毁发送音频帧线程。

步骤 2 调用 uapi_adec_stop 接口停止 ADEC。

```
uapi_adec_stop(h_adec);
```

步骤 3 调用 uapi_snd_track_stop 接口停止 TRACK。

```
uapi_snd_track_stop(h_track);
```

步骤 4 调用 uapi_adp_detach_output 接口解除输入 ADP 与 ADEC 的绑定关系。

```
uapi_adp_detach_output(h_adp, h_adec);
```

步骤 5 调用 uapi_adp_destroy 接口销毁输入 ADP 通路。

```
uapi_adp_destroy(h_adp);
```

步骤 6 调用 uapi_adec_detach_output 接口解除 ADEC 与 TRACK 的绑定关系。

```
uapi_adec_detach_output(h_adec, h_track);
```

步骤 7 调用 uapi_adec_destroy 接口销毁 ADEC。

```
uapi_adec_destroy(h_adec);
```

步骤 8 调用 uapi_snd_destroy_track 接口销毁 TRACK。如果创建了多路音频解码，则需要为每一路执行[步骤 1](#)~[步骤 8](#)。

```
uapi_snd_destroy_track(h_track);
```

步骤 9 调用 uapi_snd_close 接口关闭 SOUND。

```
uapi_snd_close(h_snd);
```

步骤 10 调用 uapi_adec_deinit 接口去初始化 ADEC 模块。

```
uapi_adec_deinit();
```

步骤 11 调用 uapi_snd_deinit 接口去初始化 SOUND 模块。

```
uapi_snd_deinit();
```

步骤 12 调用 uapi_adp_deinit 接口去初始化 ADP 模块。

```
uapi_adp_deinit();
```

----结束

5.3 注意事项

请严格按照“[5.2.2 解码播放场景](#)”中描述的步骤顺序进行接口调用。

Draft

6 AENC

6.1 概述

6.2 开发指引

6.3 注意事项

6.1 概述

AENC (Audio Encoder) 即音频编码模块。AENC 模块接收前级模块或用户发送的音频数据，根据用户选择的编码器生成不同格式的码流。生成的码流数据返回给用户使用，例如通过网络发送到远端解码播放。

AENC 模块的数据来源主要有音频输入 (AI)、音频 SEA、音频蓝牙输出 (BT) 以及上层应用。AENC 对输入数据进行编码后输送给后级模块。

说明

AENC 模块支持 MP3、SBC、MSBC、LC3、PCM 等编码类型。

同时最多允许打开 1 个 AENC 实例。

6.2 开发指引

6.2.1 接口说明

AENC 模块实现了以下功能：

- AENC 模块系统管理功能，实现初始化、去初始化等功能，该功能模块提供以下 API：

- uapi_aenc_init: 初始化 AENC 模块。
- uapi_aenc_deinit: 去初始化 AENC 模块。
- AENC 模块实例管理功能, 该功能模块提供以下 API:
 - uapi_aenc_create: 创建一路音频编码实例。
 - uapi_aenc_destroy: 销毁一路音频编码实例。
 - uapi_aenc_get_attr: 获取音频编码实例的属性。
 - uapi_aenc_set_attr: 设置音频编码实例的属性。
 - uapi_aenc_start: 启动音频编码实例。
 - uapi_aenc_stop: 停止音频编码实例。
- AENC 与其他模块级联功能, 该功能模块提供以下 API:
 - uapi_aenc_attach_output: 为音频编码实例绑定一个输出。
 - uapi_aenc_detach_output: 为音频编码实例解绑一个输出。
- AENC 事件上报功能, 该功能模块提供以下 API:
 - uapi_aenc_register_event_proc: 注册 AENC 事件处理回调函数, 事件包括编码处理和获取时延。

编码器实例属性如下:

- codec_id: 每种音频格式都唯一对应的 ID。
- max_trans_unit_size: 最大帧长。
- param: 其他参数, 主要包括声道数、位宽、采样率、私有数据地址以及长度。

📖 说明

uapi_aenc_register_event_proc 暂不支持。

6.2.2 用户送帧场景

用户根据自己的实际需求, 将需要编码的音频帧通过 ADP 实例送到 AENC 模块进行编码, 并通过另一个 ADP 实例获取编码后的码流。

📖 说明

在调用其他 AENC 接口之前, 必须先对 AENC 模块初始化; 不再使用 AENC 模块时, 须将 AENC 模块去初始化。

用户送帧初始化步骤

用户送帧初始化步骤如下:

步骤 1 调用 uapi_adp_init 接口初始化 ADP 模块。

```
uapi_adp_init();
```

步骤 2 调用 uapi_aenc_init 接口初始化 AENC 模块。

```
uapi_aenc_init();
```

步骤 3 调用 uapi_adp_get_default_attr 接口获取输出 ADP 的默认属性。

```
uapi_adp_attr adp_attr;  
uapi_adp_get_def_attr(&adp_attr);
```

步骤 4 调用 uapi_adp_create 接口创建一路输出 ADP，用于获取 AENC 的输出。

```
td_handle h_adp_out = 0;  
uapi_adp_create(&h_adp_out, &adp_attr);
```

步骤 5 调用 uapi_aenc_create 接口创建一路 AENC，用于音频帧的编码。

```
td_handle h_aenc = 0;  
uapi_aenc_create(&h_aenc);
```

步骤 6 调用 uapi_aenc_get_attr 接口获取 AENC 的属性。

```
uapi_aenc_attr aenc_attr;  
uapi_aenc_get_attr(h_aenc, &aenc_attr);
```

步骤 7 修改属性，调用 uapi_aenc_set_attr 接口设置新属性。

```
aenc_attr.codec_id = UAPI_ACODEC_ID_MSBC;  
aenc_attr.param.interleaved = TD_TRUE;  
aenc_attr.param.sample_rate = UAPI_AUDIO_SAMPLE_RATE_16K;  
aenc_attr.param.channels = UAPI_AUDIO_CHANNEL_1;  
aenc_attr.param.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;  
aenc_attr.param.samples_per_frame = aenc_attr.param.sample_rate / 100; /* 10ms */  
aenc_attr.param.private_data = TD_NULL;  
aenc_attr.param.private_data_size = 0;  
uapi_aenc_set_attr(h_aenc, &aenc_attr);
```

步骤 8 调用 uapi_aenc_attach_output 接口将 AENC 的输出设置为输出 ADP。

```
uapi_aenc_attach_output(h_aenc, h_adp_out);
```

步骤 9 调用 uapi_adp_get_default_attr 接口获取输入 ADP 的默认属性。

```
uapi_adp_get_def_attr(&adp_attr);
```

步骤 10 调用 uapi_adp_create 接口创建一路输入 ADP，用于给 AENC 送入音频数据。

```
td_handle h_adp_in = 0;
```

```
uapi_adp_create(&h_adp_in, &adp_attr);
```

步骤 11 调用 `uapi_adp_attach_output` 接口将输入 ADP 的输出设置为 AENC。

```
uapi_adp_attach_output(h_adp_in, h_aenc);
```

步骤 12 调用 `uapi_aenc_start` 接口启动 AENC 进行编码。

```
uapi_aenc_start(h_aenc);
```

步骤 13 创建一个送音频帧线程，反复调用 `uapi_adp_send_frame` 接口向输入 ADP 送入 PCM 数据。

```
td_s32 ret;
td_bool task_active = TD_TRUE;
uapi_audio_frame input_frame;
while (task_active) {
    read_frame(&input_frame); /* read_frame需用户自行开发，用于获取音频帧 */
    while (task_active) {
        ret = uapi_adp_send_frame(h_adp, &frame);
        if (ret != EXT_SUCCESS) {
            continue;
        }
        break;
    }
}
```

步骤 14 创建一个接收码流线程，调用 `uapi_adp_acquire_stream` 接口从输出 ADP 获取编码后的码流，使用完之后调用 `uapi_adp_release_stream` 接口释放该码流。

```
td_s32 ret;
td_bool task_active = TD_TRUE;
uapi_stream_buf output_stream;
while (task_active) {
    while (task_active) {
        ret = uapi_adp_acquire_stream(h_adp, &output_stream);
        if (ret != EXT_SUCCESS) {
            continue;
        }
        process_stream(output_stream); /* process_stream需用户自行开发，用于处理编码后的码流 */
        ret = uapi_adp_release_stream(h_adp, &output_stream);
        if (ret != EXT_SUCCESS) {
            continue;
        }
    }
}
```

```
        break;
    }
}
```

----结束

📖 说明

AENC 没有音频数据接口，需要创建一路输入 ADP 和一路输出 ADP，并且将 AENC 绑定到输入 ADP 的后级，同时将输出 ADP 绑定到 AENC 的后级。

用户送帧去初始化步骤

用户送帧去初始化步骤如下：

步骤 1 销毁接收码流线程。

步骤 2 销毁送音频帧线程。

步骤 3 调用 uapi_aenc_stop 接口停止 AENC。

```
uapi_aenc_stop(h_aenc);
```

步骤 4 调用 uapi_adp_detach_output 接口解除输入 ADP 与 AENC 的绑定关系。

```
uapi_adp_detach_output(h_adp_in, h_aenc);
```

步骤 5 调用 uapi_adp_destroy 接口销毁输入 ADP 通路。

```
uapi_adp_destroy(h_adp_in);
```

步骤 6 调用 uapi_aenc_detach_output 接口解除 AENC 与输出 ADP 的绑定关系。

```
uapi_aenc_detach_output(h_aenc, h_adp_out);
```

步骤 7 调用 uapi_aenc_destroy 接口销毁 AENC。

```
uapi_aenc_destroy(h_aenc);
```

步骤 8 调用 uapi_adp_destroy 接口销毁输出 ADP 通路。

```
uapi_adp_destroy(h_adp_out);
```

步骤 9 调用 uapi_aenc_deinit 接口去初始化 AENC 模块。

```
uapi_aenc_deinit();
```

步骤 10 调用 uapi_adp_deinit 接口去初始化 ADP 模块。

```
uapi_adp_deinit();
```

----结束

6.3 注意事项

- 绑定输入源/输出源需要在编码启动之前，取消绑定关系需要在停止编码之后进行。
- 在调用 `uapi_aenc_set_attr` 接口设置编码属性前必须先停止编码。配置好编码属性之后，再启动编码。
- 在调用 `uapi_aenc_set_attr` 接口动态配置编码属性时要注意传入属性的合法性。合法性包括：各个参数在规定的取值范围内；非动态配置的属性不能予以更改。建议在调用 `uapi_aenc_set_attr` 接口前先调用 `uapi_aenc_get_attr` 接口获取到当前编码器属性，再针对要修改的属性值进行修改。

7 AI

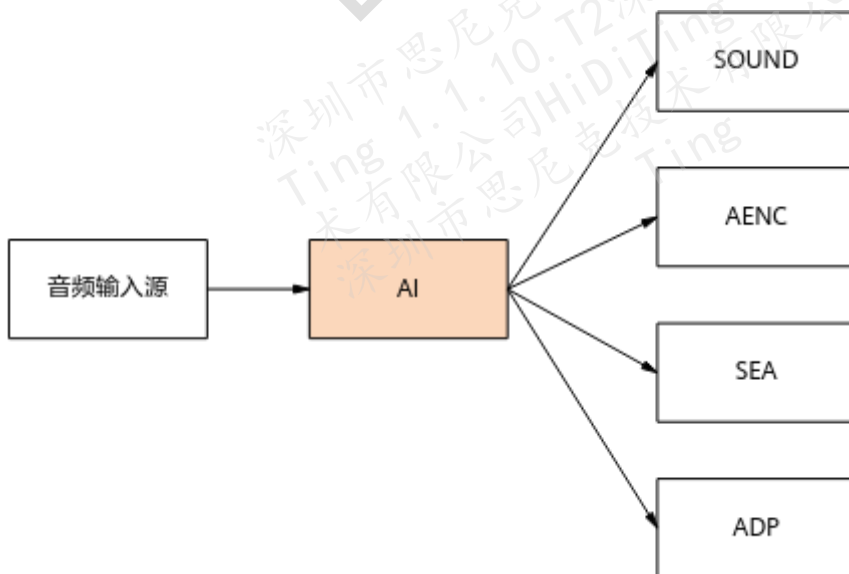
7.1 概述

7.2 开发指引

7.1 概述

AI (Audio Input) 模块提供音频采集功能。AI 采集来自前级模块的音频，连接后级模块进行音频编码或者保存数据用作其他用途。AI 的数据流如图 7-1 所示。

图7-1 AI 的数据流处理过程



AI 可以通过 I2S 接口接收音频数据存入指定的缓冲区，而后推送给 SEA 或 AENC 或 SOUND；或者保存数据，进行后级处理。AI 向应用提供基本的音频采集接口。

重要概念：

- AI_PORT：音频物理输入端口，包括 ADC、DMIC、I2S 等。支持对 PORT 进行静音、音量、声道模式设置。
- 音量：支持音量设置，即硬件端口的音量设置。AI 音量设置是直接设置逻辑寄存器，反映了逻辑的音量处理能力。
 - PDM 端口的音量设置范围是 -120dB ~ +60dB，步长为 1dB。
 - ADC 端口的音量设置范围是 0dB ~ +50dB，步长为 2dB。
 - LPADC 端口的音量设置范围是 0dB ~ +50dB，步长为 3dB。LPADC 端口的音量设置范围分为两个区间：
 - 当设置值大于等于 0dB 且小于 26dB 时，以 0dB 为起点，步长为 3dB。
 - 当设置值大于等于 26dB 且小于等于 50dB 时，以 26dB 为起点，步长为 3dB。

说明

当音量设置在范围内，但不满足步长限制时，实际值为小于且最接近设置值的满足步长限制的值，如：

- PDM 端口的音量设置为 10.5dB 时，实际值为 10dB。
- ADC 端口的音量设置为 23dB 时，实际值为 22dB。
- LPADC 端口的音量设置为 25dB 时，实际值为 24dB。
- LPADC 端口的音量设置为 36dB 时，实际值为 35dB。
- VAD (Voice Activity Detection)：支持语音检测，按检测方法分为模拟语音检测模块 (AVAD) 和数字语音检测模块 (DVAD/MAD)。
- 回声参考：支持回声拼接，输出端口的音频数据内环回到输入端口与原始数据拼接，作为参考信号同步录制。

说明

同时最多允许打开 2 个 AI 实例。

7.2 开发指引

7.2.1 接口说明

AI 模块实现了以下功能：

- 音频采集初始化和去初始化功能，该功能模块提供以下 API：

- uapi_ai_init: AI 初始化。
- uapi_ai_deinit: AI 去初始化。
- 音频采集管理功能, 包括 AI 模块的创建、销毁、属性获取和设置, 该功能模块提供以下 API:
 - uapi_ai_get_default_attr: 获取 AI 默认属性。
 - uapi_ai_open: 打开指定的输入端口, 并创建一路 AI 实例。
 - uapi_ai_close: 销毁一路 AI 实例, 若 AI 实例所在输入端口没有任何其他 AI 实例时, 关闭该输入端口。
 - uapi_ai_get_attr: 获取 AI 属性。
 - uapi_ai_set_attr: 设置 AI 属性, 设置前需停止 AI。不同的 AI 实例可能会设置同一个输入端口, 同个输入端口的所有 AI 实例中, 任一实例最近一次设置生效的属性, 会自动同步到其他所有 AI 实例。
 - uapi_ai_get_mute: 获取 AI 静音状态。
 - uapi_ai_set_mute: 设置 AI 静音状态, 设置前需停止 AI。
 - uapi_ai_get_volume: 获取 AI 音量。
 - uapi_ai_set_volume: 设置 AI 音量, 设置前需停止 AI。
- 音频语音检测功能, 该功能模块提供以下 API:
 - uapi_ai_reset_vad: 复位 VAD, 进入下一个检测周期。
- 音频采集事件处理功能, 该功能模块提供以下 API:
 - uapi_ai_register_event_proc: 注册事件处理函数, 处理语音检测成功、语音检测超时、数据传输完成等事件。
- 音频采集控制功能, 包括 AI 模块的启动、停止, 该功能模块提供以下 API:
 - uapi_ai_start: 启动 AI 实例。如果 AI 实例所在的输入端口未使能时, 将自动使能输入端口。
 - uapi_ai_stop: 停止 AI 实例。如果 AI 实例所在的输入端口上没有其他 AI 实例或者所有 AI 实例都停止时, 输入端口将自动停止 (不使能)。
- AI 与其他模块级联功能, 该功能模块提供以下 API:
 - uapi_ai_attach_output: 为 AI 绑定一个输出。
 - uapi_ai_detach_output: 解除 AI 与输出的绑定关系。

AI 的属性参数及其默认配置如表 7-1 所示。

表7-1 AI 属性参数说明

属性名称	默认配置	说明
uapi_ai_port	无（用户设置）	音频输入端口。支持 I2S0/ I2S1/ I2S2、ADC0/LPADC0（AMIC）、PDM0（DMIC），其他接口不支持。
uapi_ai_gain	无（端口有差异）	音频输入端口音量配置。 - 内置 ADC/LPADC：0 ~ +50 - PDM：-120 ~ +60
uapi_ai_vad_type	UAPI_AI_VAD_MAD	音频输入端口语音检测模块类型（模拟语音检测模块（AVAD）/数字语音检测模块（DVAD/MAD））。
uapi_ai_attr.pcm_attr.channels	UAPI_AUDIO_CHANNEL_2	音频输入声道数。
uapi_ai_attr.pcm_attr.bit_depth	UAPI_AUDIO_BIT_DEPTH_16	音频输入位宽。
uapi_ai_attr.pcm_attr.sample_rate	UAPI_AUDIO_SAMPLE_RATE_16K	音频输入采样率。
uapi_ai_attr.pcm_attr.sample_per_frame	10ms 的采样点数	数据上报/处理帧长。
uapi_ai_attr.ref_attr.enable	TD_FALSE	回声参考通道开关。
uapi_ai_attr.ref_attr.port	UAPI_SND_OUT_PORT_DAC0	回声参考通道数据来源。
uapi_ai_attr.vad_attr.enable	TD_FALSE	AI VAD 功能开关。
uapi_ai_attr.vad_attr.attr	无（用户设置）	AI VAD 配置。
uapi_ai_attr.port_attr.adc.rx_type	UAPI_AI_RX_DEFAULT（AIAO_RX）	AI ADC 端口的接收通道类型（AIAO_RX/MAD_RX）。
uapi_ai_attr.port_attr.adc.mode	UAPI_AI_ADC_NORMAL_MODE	AI ADC 端口的工作模式（暂不支持设置）。
uapi_ai_attr.	UAPI_AI_RX_DEFAULT	AI PDM 端口的接收通道类型

属性名称	默认配置	说明
port_attr.pdm.rx_type	ULT (AIAO_RX)	(AIAO_RX/MAD_RX)。
uapi_ai_attr.port_attr.pdm.i2s_attr	无 (用户设置)	AI PDM 端口的 I2S 配置 (同下 I2S 配置)。
uapi_ai_attr.port_attr.i2s.i2s_attr	无 (用户设置)	AI I2S 端口的 I2S 配置。
uapi_ai_attr.port_attr.i2s.i2s_attr.master	TD_TRUE	AI I2S 端口的主从模式配置。
uapi_ai_attr.port_attr.i2s.i2s_attr.i2s_mode	UAPI_AUDIO_I2S_S TD_MODE	AI I2S 端口的接口模式 (暂不支持设置)。
uapi_ai_attr.port_attr.i2s.i2s_attr.mclk	UAPI_AUDIO_I2S_M CLK_768_FS	AI I2S 端口的采样率分频系数, mclk 固定为 12.288MHz。
uapi_ai_attr.port_attr.i2s.i2s_attr.bclk	UAPI_AUDIO_I2S_B CLK_12_DIV	AI I2S 端口的位时钟分频系数, mclk 固定为 12.288MHz。
uapi_ai_attr.port_attr.i2s.i2s_attr.bit_depth	UAPI_AUDIO_BIT_D EPH_16	AI I2S 端口的输入位宽。
uapi_ai_attr.port_attr.i2s.i2s_attr.channels	UAPI_AUDIO_CHAN NEL_2	AI I2S 端口的输入声道数。
uapi_ai_attr.port_attr.i2s.i2s_attr.pcm_sample_rise_edge	TD_TRUE	AI I2S 端口的采样沿的极性, 仅 PCM 模式有效。
uapi_ai_attr.port_attr.i2s.i2s_attr.pcm_delay_cycle	UAPI_AUDIO_I2S_P CM_0_DELAY	AI I2S 端口的数据有效延迟周期, 仅 PCM 模式有效。

7.2.2 音频录制场景

音频录制场景是指将 MIC 数据通过接收通道送至 AI 模块, 然后 AI 模块对来自 MIC 的 PCM 数据进行编码存储。录制过程中可以对录制端口进行音量设置、静音设置等。

说明

在调用其他 AI 接口之前，必须先对 AI 模块初始化；不再使用 AI 模块时，须将 AI 模块去初始化。

音频录制初始化步骤

音频录制初始化步骤如下：

步骤 1 调用 `uapi_adp_init` 接口初始化 ADP 模块。

```
uapi_adp_init();
```

步骤 2 调用 `uapi_ai_init` 接口初始化 AI 模块。

```
uapi_ai_init();
```

步骤 3 调用 `uapi_adp_get_def_attr` 接口获取 ADP 的默认属性。

```
uapi_adp_attr adp_attr;  
uapi_adp_get_def_attr(&adp_attr);
```

步骤 4 调用 `uapi_adp_create` 接口创建一路 ADP。

```
td_handle h_adp = 0;  
uapi_adp_create(&h_adp, &adp_attr);
```

步骤 5 设置默认 AI 端口，调用 `uapi_ai_get_default_attr` 接口获取 AI 端口默认属性。

```
uapi_ai_port ai_port = UAPI_AI_PORT_PDM0;  
uapi_ai_attr ai_attr;  
uapi_ai_get_default_attr(ai_port, &ai_attr);
```

步骤 6 按照需要修改 AI 端口属性。

```
ai_attr.pcm_attr.sample_rate = UAPI_AUDIO_SAMPLE_RATE_16K;  
ai_attr.pcm_attr.channels = UAPI_AUDIO_CHANNEL_1;  
ai_attr.pcm_attr.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;  
ai_attr.pcm_attr.samples_per_frame = ai_attr.pcm_attr.sample_rate / 100; /* 10ms */  
ai_attr.port_attr.pdm.rx_type = UAPI_AI_RX_DEFAULT;  
ai_attr.port_attr.pdm.i2s_attr.channels = UAPI_AUDIO_CHANNEL_1;  
ai_attr.port_attr.pdm.i2s_attr.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;
```

步骤 7 调用 `uapi_ai_open` 接口打开音频输入设备。

```
td_handle h_ai = 0;  
uapi_ai_open(&h_ai, ai_port, &ai_attr);
```

步骤 8 调用 `uapi_ai_register_event_proc` 接口注册 AI 事件处理函数（VAD 事件等）。

```
td_void *context;  
uapi_ai_register_event_proc(h_ai, ai_event_proc, context); /* ai_event_proc 为用户自行注册的事件  
处理函数 */
```

步骤 9 调用 uapi_ai_attach_output 接口将 AI 的输出设置为 ADP。

```
uapi_ai_attach_output(h_ai, h_adp);
```

步骤 10 调用 uapi_ai_start 接口启动 AI 录制。

```
uapi_ai_start(h_ai);
```

步骤 11 创建一个接收音频帧线程，反复调用 uapi_adp_acquire_frame 接口从 ADP 获取来自 MIC 的 PCM 数据，保存后再调用 uapi_adp_release_frame 接口释放数据。

```
td_s32 ret;  
td_bool task_active = TD_TRUE;  
uapi_audio_frame frame;  
while (task_active) {  
    while (task_active) {  
        ret = uapi_adp_acquire_frame(h_adp, &frame);  
        if (ret != EXT_SUCCESS) {  
            continue;  
        }  
        save_frame(frame); /* save_frame需用户自行开发，用于保存音频帧 */  
        ret = uapi_adp_release_frame(h_adp, &frame);  
        if (ret != EXT_SUCCESS) {  
            continue;  
        }  
        break;  
    }  
}
```

----结束

📖 说明

AI 没有音频数据出口，需要创建一路 ADP，并且将 ADP 绑定到 AI 的输出，获取 PCM 数据。

音频录制去初始化步骤

音频录制去初始化步骤如下：

步骤 1 销毁接收音频帧线程。

步骤 2 调用 uapi_ai_stop 接口停止 AI。

```
uapi_ai_stop(h_ai);
```

步骤 3 调用 uapi_ai_detach_output 接口解除 AI 与 ADP 的绑定关系。

```
uapi_ai_detach_output(h_ai, h_adp);
```

步骤 4 调用 uapi_ai_close 接口关闭 AI。

```
uapi_ai_close(h_ai);
```

步骤 5 调用 uapi_adp_destroy 接口销毁 ADP 通路。

```
uapi_adp_destroy(h_adp);
```

步骤 6 调用 uapi_ai_deinit 接口去初始化 AI 模块。

```
uapi_ai_deinit();
```

步骤 7 调用 uapi_adp_deinit 接口去初始化 ADP 模块。

```
uapi_adp_deinit();
```

----结束

8 SEA

8.1 概述

8.2 开发指引

8.3 注意事项

8.1 概述

SEA (Speech Enhancement and Artificial Intelligence) 即语音增强与智能。SEA 是一个功能强大的智能语音框架，整合了各种类型的语音增强和语音智能算法，还支持集成第三方的智能语音解决方案；向开发者提供统一的 UAPI 接口，创建满足各种语音业务场景的实例。

- 开发者可以通过 UAPI 接口创建满足语音唤醒、语音识别、语音通话或人声增强等业务场景的单一实例。
- 开发者可以通过 UAPI 接口创建多个功能单一的实例，并通过级联方式搭建链路来满足各种不同的语音业务场景。
- 算法提供方可以基于 SEA 算法插件接口集成特性算法到框架给开发者使用。SEA 算法插件接口描述请参见《HiDiTingV100 音频算法插件 开发指南》。

说明

SEA 当前只提供近场的语音唤醒、命令词和语音通话业务所需算法，其他场景业务（如远场语音、语音识别、声纹识别和人声增强等）所需算法或解决方案将在后续迭代版本中支持。

同时最多允许打开 2 个 SEA 实例。

8.2 开发指引

8.2.1 接口说明

SEA 模块实现了以下功能：

- SEA 模块系统管理功能，实现初始化、去初始化等功能，该功能模块提供以下 API：
 - uapi_sea_init：初始化 SEA 模块。
 - uapi_sea_deinit：去初始化 SEA 模块。
- SEA 模块引擎管理功能，实现引擎库动态加载、卸载等功能，该功能模块提供以下 API：
 - uapi_sea_load_engine：加载指定的引擎库。
 - uapi_sea_unload_engine：卸载指定的引擎库（该接口暂不支持）。
 - uapi_sea_get_engine_caps：获取指定引擎库的能力集合。
- SEA 模块实例管理功能，该功能模块提供以下 API：
 - uapi_sea_get_default_attr：基于指定的能力组合获取默认属性。
 - uapi_sea_create：创建一个指定能力的语音处理实例。
 - uapi_sea_destroy：销毁指定的语音处理实例。
 - uapi_sea_get_attr：获取语音处理实例的属性。
 - uapi_sea_set_attr：设置语音处理实例的属性（该接口暂不支持）。
 - uapi_sea_start：启动语音处理实例。
 - uapi_sea_stop：停止语音处理实例。
 - uapi_sea_get_afe_attr：获取语音增强引擎的属性（语音实例配置语音增强能力时，该接口才有效）。
 - uapi_sea_set_afe_attr：设置语音增强引擎的属性（语音实例配置语音增强能力时，该接口才有效）。
 - uapi_sea_get_aai_attr：获取语音 AI 引擎的属性（语音实例配置语音 AI 能力时，该接口才有效）。
 - uapi_sea_set_aai_attr：设置语音 AI 引擎的属性（语音实例配置语音 AI 能力时，该接口才有效）。
 - uapi_sea_get_param：获取算法参数（该接口暂不支持）。
 - uapi_sea_set_param：设置算法参数（该接口暂不支持）。

- uapi_sea_get_phrase_count: 获取可识别的短语数量（语音实例配置命令词能力时，该接口才有效）。
- uapi_sea_get_phrase_sets: 获取可识别的短语集合（语音实例配置命令词能力时，该接口才有效）。
- uapi_sea_get_entity_count: 获取可识别的实体数量（语音特征识别能力接口，该接口暂不支持）。
- uapi_sea_get_entity_sets: 获取可识别的实体集合（语音特征识别能力接口，该接口暂不支持）。
- uapi_sea_start_vid_enroll: 启动语音 ID 注册（声纹识别能力接口，该接口暂不支持）。
- uapi_sea_stop_vid_enroll: 停止语音 ID 注册（声纹识别能力接口，该接口暂不支持）。
- uapi_sea_add_vid_entity: 新增语音 ID（声纹识别能力接口，该接口暂不支持）。
- uapi_sea_remove_vid_entity: 删除语音 ID（声纹识别能力接口，该接口暂不支持）。
- SEA 与其他模块级联功能，该功能模块提供以下 API：
 - uapi_sea_attach_output: 为语音处理实例绑定一个输出。
 - uapi_sea_detach_output: 为语音处理实例解除一个输出。
- SEA 事件上报回调注册功能，该功能模块提供以下 API：
 - uapi_sea_register_event_proc: 注册 SEA 事件处理回调函数（主要支持语音处理状态、VAD 起止、识别结果等事件上报）。
- SEA 能力集合配置项如表 8-1 所示（单个实例最大可配置 1 个语音增强引擎和 3 个语音 AI 引擎）。

表8-1 SEA 能力集合配置项

配置项	默认值	说明
uapi_sea_eng_se l.afe.type	无	语音增强类型：语音增强、通话增强、人声增强（耳机）等。
uapi_sea_eng_se l.afe.lib_id	无	语音增强引擎 ID：提供语音增强能力的引擎 ID，有系统内置的引擎 ID 和预留给开发者集成第三方的引擎 ID。

配置项	默认值	说明
uapi_sea_eng_sel.aai.type	无	语音 AI 类型：唤醒词（KWS）、语音识别（ASR）、声纹识别（VID）、场景识别（ASD）等。
uapi_sea_eng_sel.aai.lib_id	无	语音 AI 引擎 ID：提供语音 AI 能力的引擎 ID，有系统自动的引擎 ID 和预留给开发者集成第三方的引擎 ID。

说明

单个语音处理实例最大可配置 1 个语音增强引擎和 3 个语音 AI 引擎。若使用场景超过该能力集时，可创建多个语音处理实例，并通过级联多个语音处理实例来满足场景需求。

- SEA 实例属性主要如表 8-2 所示。

表8-2 SEA 实例属性

属性项	默认值	说明
uapi_sea_attr.in_buf_dur_ms	与具体的引擎相关	输入的语音数据缓存时长，单位：ms。
uapi_sea_attr.in_pcm	与具体的引擎相关	输入的语音信号 PCM 格式：采样率、位宽、声道数及每帧采样点数。
uapi_sea_attr.ref_pcm	与具体的引擎相关	输入的参考信号 PCM 格式：采样率、位宽、声道数及每帧采样点数。
uapi_sea_attr.out_pcm	与具体的引擎相关	输出处理后的语音信号 PCM 格式：采样率、位宽、声道数及每帧采样点数。 单个引擎可能输出多路数据，开发者按需配置即可。

- SEA 语音增强引擎属性如表 8-3 所示。

表8-3 SEA 语音增强引擎属性

属性项	默认值	说明
-----	-----	----

属性项	默认值	说明
uapi_sea_afe_at tr.type	无	语音增强类型：语音增强、通话增强、人声增强（耳机）等。
uapi_sea_afe_at tr.mode	无	工作模式：正常模式、低功耗模式、高性能模式。
uapi_sea_afe_at tr.u.see	与具体的引擎相关	语音增强属性配置，提供开关增强算法特性的选项。
uapi_sea_afe_at tr.u.vqe	与具体的引擎相关	通话增强属性配置，提供开关通话算法特性的选项。
uapi_sea_afe_at tr.u.ahe	与具体的引擎相关	通话人声属性配置，提供开关通话算法特性的选项。

- SEA 语音 AI 引擎属性如表 8-4 所示。

表8-4 SEA 语音 AI 引擎属性

属性项	默认值	说明
uapi_sea_aai_at tr.type	无	语音 AI 类型：唤醒词（KWS）、语音识别（ASR）、声纹识别（VID）、场景识别（ASD）等。
uapi_sea_aai_at tr.mode	无	工作模式：正常模式、低功耗模式、高性能模式。
uapi_sea_aai_at tr.u.kws	与具体的引擎相关	语音唤醒属性配置：灵敏度、语音最大时长、VAD 端点门限。
uapi_sea_aai_at tr.u.asr	与具体的引擎相关	语音识别属性配置：NLP 使能、语音最大时长、VAD 端点门限。
uapi_sea_aai_at tr.u.vid	与具体的引擎相关	声纹识别属性配置：灵敏度、语音最小时长、最大识别数。
uapi_sea_aai_at tr.u.asd	与具体的引擎相关	场景识别属性配置：时延。单位：ms。

说明

调用 `uapi_sea_create` 时需要对参数范围进行如下限制:

- `in_buf_dur_ms`: 未使用, 无需限制。
- 当设置 1mic 算法库功能时:
 - `in_pcm`:
`channels`: 1
`bit_depth`: 16
`sample_rate`: 16k
`samples_per_frame`: 10ms/20ms 的采样点数。
 - `ref_pcm`:
`channels`: 0,1
`bit_depth`: 16
`sample_rate`: 16k
`samples_per_frame`: 320
 - `out_pcm`: 只支持 `SEA_OUTPUT_ASR_SRC` 和 `SEA_OUTPUT_KWS_SRC` 通道。
`SEA_OUTPUT_ASR_SRC`
`channels`: 0,1
`bit_depth`: 16
`sample_rate`: 16k
`samples_per_frame`: 320
`SEA_OUTPUT_KWS_SRC`
`channels`: 0,1
`bit_depth`: 16
`sample_rate`: 16k
`samples_per_frame`: 320
- 当设置 phs 算法库功能时:
 - `in_pcm`:
`channels`: 1
`bit_depth`: 16
`sample_rate`: 16k
`samples_per_frame`: 10ms/20ms 的采样点数
 - `ref_pcm`: 未使用, 无需限制。
 - `out_pcm`: 未使用, 无需限制。

8.2.2 数据结构

以下为 SEA 模块部分数据结构的参数说明和范围限制：

- uapi_sea_lib_id
 - UAPI_SEA_LIB_NULL：无效的引擎库。
 - UAPI_SEA_LIB_SEE：语音增强引擎库。
 - UAPI_SEA_LIB_VQE：语音通话引擎库。
 - UAPI_SEA_LIB_AHE：人声增强引擎库。
 - UAPI_SEA_LIB_KWS：唤醒词或短语库。
 - UAPI_SEA_LIB_ASR：语音识别库。
 - UAPI_SEA_LIB_VID：声纹识别库（暂不支持）。
 - UAPI_SEA_LIB_ASD：环境声检测库（暂不支持）。
 - UAPI_SEA_LIB_HAID：助辅听算法库（暂不支持）。
 - UAPI_SEA_LIB_EXT0：外部扩展库。
 - UAPI_SEA_LIB_EXT1：外部扩展库。
 - UAPI_SEA_LIB_EXT2：外部扩展库。
 - UAPI_SEA_LIB_EXT3：外部扩展库。
- uapi_sea_afe_attr
 - type：语音前处理类型（该参数未使用）。
 - mode：工作模式。
 - u.see.options：[0, 15]
 - u.vqe.options：（算法库不支持配置）。
 - u.ahe.volume：（算法库不支持配置）。
- uapi_sea_aai_attr
 - type：语音智能类型（该参数未使用）。
 - mode：工作模式。
 - u.kws：唤醒词或短语属性。
 - sensitivity：灵敏度
参数范围：[0, 100]
 - max_dura_ms：命令词或短语最大时长（算法库不支持配置）。
 - vad_bgn_thres：VAD 起始门限（算法库不支持配置）。

- vad_end_thres: VAD 结束门限 (算法库不支持配置)。
- keyword: 唤醒词或短语列表 (算法库不支持配置)。
- u.asr: 语音识别属性 (算法库不支持配置)。
- u.vid: 声纹识别属性 (算法库不支持配置)。
- u.asd: 环境声检测属性 (算法库不支持配置)。

8.2.3 语音识别场景

📖 说明

创建一个语音处理实例，用于处理从音频输入设备采集到的语音数据，并将识别结果上报给应用。

当前支持的命令词有以下 20 个：

- { 1, "打开运动"},
- { 2, "接听电话"},
- { 3, "拒接电话"},
- { 4, "上一首"},
- { 5, "下一首"},
- { 6, "调大音量"},
- { 7, "调小音量"},
- { 8, "开始播放"},
- { 9, "停止播放"},
- {10, "打开天气"},
- {11, "打开手电筒"},
- {12, "打开血氧"},
- {13, "打开闹钟"},
- {14, "支付宝支付"},
- {15, "微信支付"},
- {16, "打开心率"},
- {17, "打开遥控拍照"},
- {18, "打开导航"},
- {19, "导航回家"},
- {20, "导航去公司"}

语音识别初始化步骤

语音识别初始化步骤如下：

步骤 1 调用 uapi_ai_init 接口初始化音频输入设备。

```
uapi_ai_init();
```

步骤 2 调用 uapi_sea_init 接口初始化 SEA 模块。

```
uapi_sea_init();
```

步骤 3 设置默认 AI 端口，调用 uapi_ai_get_default_attr 接口获取音频输入设备默认属性。

```
uapi_ai_port ai_port = UAPI_AI_PORT_PDM0;  
uapi_ai_attr ai_attr;  
uapi_ai_get_default_attr(ai_port, &ai_attr);
```

步骤 4 修改属性，调用 uapi_ai_open 接口打开音频输入设备。

```
ai_attr.pcm_attr.sample_rate = UAPI_AUDIO_SAMPLE_RATE_16K;  
ai_attr.pcm_attr.channels = UAPI_AUDIO_CHANNEL_1;  
ai_attr.pcm_attr.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;  
ai_attr.pcm_attr.samples_per_frame = ai_attr.pcm_attr.sample_rate / 100; /* 10ms */  
ai_attr.port_attr.pdm.rx_type = UAPI_AI_RX_DEFAULT;  
ai_attr.port_attr.pdm.i2s_attr.channels = UAPI_AUDIO_CHANNEL_1;  
ai_attr.port_attr.pdm.i2s_attr.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;  
td_handle h_ai = 0;  
uapi_ai_open(&h_ai, ai_port, &ai_attr);
```

步骤 5 调用 uapi_sea_eng_sel 配置语音增强和语音识别选项。

```
uapi_sea_eng_sel sea_eng;  
(td_void)memset_s(&sea_eng, sizeof(sea_eng), 0, sizeof(sea_eng));  
sea_eng.afe.type = UAPI_SEA_AFE_SEE;  
sea_eng.afe.lib_id = UAPI_SEA_LIB_SEE;  
sea_eng.aai[0].type = UAPI_SEA_AAI_KWS;  
sea_eng.aai[0].lib_id = UAPI_SEA_LIB_KWS;
```

步骤 6 调用 uapi_sea_load_engine 接口加载 SEA 模块语音增强引擎库。

```
uapi_sea_load_engine(UAPI_SEA_LIB_SEE, "imedia_2mic");
```

步骤 7 调用 uapi_sea_load_engine 接口加载 SEA 模块语音识别引擎库。

```
uapi_sea_load_engine(UAPI_SEA_LIB_KWS, "imedia_keyword");
```

步骤 8 调用 uapi_sea_get_default_attr 接口获取 SEA 默认属性。

```
uapi_sea_attr sea_attr;  
uapi_sea_get_default_attr(&sea_eng, &sea_attr);
```

步骤 9 调用 uapi_sea_create 接口创建一个语音处理实例，用于语音识别。

```
td_handle h_sea= 0;
uapi_sea_create(&h_sea, &sea_eng, &sea_attr);
```

步骤 10 调用 uapi_sea_register_event_proc 接口注册一个事件上报回调接口，用于获取识别结果。

```
td_void *context;
uapi_sea_register_event_proc(h_sea, sea_event_proc, context); /* sea_event_proc为用户自行注册
的事件处理函数 */
```

步骤 11 调用 uapi_ai_attach_output 接口将语音处理实例绑定到音频输入设备。

```
uapi_ai_attach_output(h_ai, h_sea);
```

步骤 12 调用 uapi_ai_start 接口启动音频输入设备。

```
uapi_ai_start(h_ai);
```

步骤 13 调用 uapi_sea_start 接口启动语音处理实例。

```
uapi_sea_start(h_sea);
```

----结束

语音识别去初始化步骤

语音识别去初始化步骤如下：

步骤 1 调用 uapi_sea_stop 接口停止语音处理实例。

```
uapi_sea_stop(h_sea);
```

步骤 2 调用 uapi_ai_stop 接口停止音频输入设备。

```
uapi_ai_stop(h_ai);
```

步骤 3 调用 uapi_ai_detach_output 接口解除音频输入设备绑定语音处理实例。

```
uapi_ai_detach_output(h_ai, h_sea);
```

步骤 4 调用 uapi_sea_destroy 接口销毁语音处理实例。

```
uapi_sea_destroy(h_sea);
```

步骤 5 调用 uapi_ai_close 接口关闭音频输入设备。

```
uapi_ai_close(h_ai);
```

步骤 6 调用 uapi_sea_unload_engine 接口卸载语音识别引擎。

```
uapi_sea_unload_engine(UAPI_SEA_LIB_KWS, "imedia_keyword");
```

步骤 7 调用 uapi_sea_unload_engine 接口卸载语音增强引擎。

```
uapi_sea_unload_engine(UAPI_SEA_LIB_SEE, "imedia_2mic");
```

步骤 8 调用 uapi_sea_deinit 接口去初始化 SEA 模块。

```
uapi_sea_deinit();
```

步骤 9 调用 uapi_ai_deinit 接口去初始化音频输入设备。

```
uapi_ai_deinit();
```

----结束

8.2.4 语音通话场景

说明

创建一个语音处理实例，用于处理从音频输入设备采集到的语音数据，并将处理后的语音数据输出给应用。

语音通话初始化步骤

语音通话初始化步骤如下：

步骤 1 调用 uapi_adp_init 接口初始化 ADP 模块。

```
uapi_adp_init();
```

步骤 2 调用 uapi_ai_init 接口初始化音频输入设备。

```
uapi_ai_init();
```

步骤 3 调用 uapi_sea_init 接口初始化 SEA 模块。

```
uapi_sea_init();
```

步骤 4 调用 uapi_sea_load_engine 接口加载 SEA 模块语音增强引擎库。

```
uapi_sea_load_engine(UAPI_SEA_LIB_SEE, "imedia_2mic");
```

步骤 5 调用 uapi_aenc_init 接口初始化 AENC 模块。

```
uapi_aenc_init();
```

步骤 6 调用 uapi_adp_get_def_attr 接口获取 ADP 的默认属性。

```
uapi_adp_attr adp_attr;  
uapi_adp_get_def_attr(&adp_attr);
```

步骤 7 调用 `uapi_adp_create` 接口创建一路 ADP。

```
td_handle h_adp = 0;
uapi_adp_create(&h_adp, &adp_attr);
```

步骤 8 调用 `uapi_aenc_create` 接口创建一路 AENC，用于音频帧的编码。

```
td_handle h_aenc = 0;
uapi_aenc_create(&h_aenc);
```

步骤 9 调用 `uapi_aenc_get_attr` 接口获取 AENC 的属性。

```
uapi_aenc_attr aenc_attr;
uapi_aenc_get_attr(h_aenc, &aenc_attr);
```

步骤 10 修改属性，调用 `uapi_aenc_set_attr` 接口设置新属性。

```
aenc_attr.codec_id = UAPI_ACODEC_ID_MSBC;
aenc_attr.param.interleaved = TD_TRUE;
aenc_attr.param.sample_rate = UAPI_AUDIO_SAMPLE_RATE_16K;
aenc_attr.param.channels = UAPI_AUDIO_CHANNEL_1;
aenc_attr.param.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;
aenc_attr.param.samples_per_frame = aenc_attr.param.sample_rate / 100; /* 10ms */
aenc_attr.param.private_data = TD_NULL;
aenc_attr.param.private_data_size = 0;
uapi_aenc_set_attr(h_aenc, &aenc_attr);
```

步骤 11 调用 `uapi_aenc_attach_output` 接口将 AENC 的输出设置为 ADP。

```
uapi_aenc_attach_output(h_aenc, h_adp);
```

步骤 12 调用 `uapi_sea_eng_sel` 配置语音增强选项。

```
uapi_sea_eng_sel sea_eng;
(td_void)memset_s(&sea_eng, sizeof(sea_eng), 0, sizeof(sea_eng));
sea_eng.afe.type = UAPI_SEA_AFE_SEE;
sea_eng.afe.lib_id = UAPI_SEA_LIB_SEE;
```

步骤 13 调用 `uapi_sea_get_default_attr` 接口获取 SEA 默认属性。

```
uapi_sea_attr sea_attr;
uapi_sea_get_default_attr(&sea_eng, &sea_attr);
```

步骤 14 修改属性，调用 `uapi_sea_create` 接口创建一个语音处理实例，用于语音增强。

```
sea_attr.in_pcm.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;
sea_attr.in_pcm.channels = UAPI_AUDIO_CHANNEL_1;
sea_attr.in_pcm.sample_rate = UAPI_AUDIO_SAMPLE_RATE_16K;
sea_attr.in_pcm.sample_per_frame = sea_attr.in_pcm.sample_rate / 100; /* 10ms */
```

```
sea_attr.ref_pcm.channels = UAPI_AUDIO_CHANNEL_1;
td_handle h_sea= 0;
uapi_sea_create(&h_sea, &sea_eng, &sea_attr);
```

步骤 15 调用 uapi_sea_attach_output 接口将 SEA 的输出设置为 AENC。

```
uapi_sea_attach_output(h_sea, UAPI_SEA_OUTPUT_ASR_SRC, h_aenc);
```

步骤 16 设置默认 AI 端口，调用 uapi_ai_get_default_attr 接口获取音频输入设备默认属性。

```
uapi_ai_port ai_port = UAPI_AI_PORT_PDM1;
uapi_ai_attr ai_attr;
uapi_ai_get_default_attr(ai_port, &ai_attr);
```

步骤 17 修改属性，调用 uapi_ai_open 接口打开音频输入设备。

```
ai_attr.pcm_attr.sample_rate = UAPI_AUDIO_SAMPLE_RATE_16K;
ai_attr.pcm_attr.channels = UAPI_AUDIO_CHANNEL_1;
ai_attr.pcm_attr.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;
ai_attr.pcm_attr.samples_per_frame = ai_attr.pcm_attr.sample_rate / 100; /* 10ms */
ai_attr.port_attr.pdm.i2s_attr.channels = UAPI_AUDIO_CHANNEL_1;
ai_attr.port_attr.pdm.i2s_attr.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;
ai_attr.ref_attr.enable = TD_TRUE;
ai_attr.ref_attr.port = UAPI_SND_OUT_PORT_DAC0;
td_handle h_ai= 0;
uapi_ai_open(&h_ai, ai_port, &ai_attr);
```

步骤 18 调用 uapi_ai_attach_output 接口将 AI 的输出设置为 SEA。

```
uapi_ai_attach_output(h_ai, h_sea);
```

步骤 19 创建一个解码播放器用于处理 ADP 中的 PCM 编码数据，可参考 [4.2.2 音频播放场景](#)。

步骤 20 调用 uapi_ai_start 接口启动音频输入设备。

```
uapi_ai_start(h_ai);
```

步骤 21 调用 uapi_sea_start 接口启动语音处理实例。

```
uapi_sea_start(h_sea);
```

步骤 22 调用 uapi_aenc_start 接口启动 AENC 进行编码。

```
uapi_aenc_start(h_aenc);
```

----结束

语音通话去初始化步骤

语音通话去初始化步骤如下：

步骤 1 调用 uapi_aenc_stop 接口停止 AENC。

```
uapi_aenc_stop(h_aenc);
```

步骤 2 调用 uapi_sea_stop 接口停止语音处理实例。

```
uapi_sea_stop(h_sea);
```

步骤 3 调用 uapi_ai_stop 接口停止音频输入设备。

```
uapi_ai_stop(h_ai);
```

步骤 4 调用 uapi_ai_detach_output 接口解除 AI 与 SEA 的绑定关系。

```
uapi_ai_detach_output(h_ai, h_sea);
```

步骤 5 调用 uapi_ai_close 接口关闭音频输入设备。

```
uapi_ai_close(h_ai);
```

步骤 6 调用 uapi_sea_detach_output 接口解除 SEA 与 AENC 的绑定关系。

```
uapi_sea_detach_output(h_sea, UAPI_SEA_OUTPUT_ASR_SRC, h_aenc);
```

步骤 7 调用 uapi_sea_destroy 接口销毁语音处理实例。

```
uapi_sea_destroy(h_sea);
```

步骤 8 调用 uapi_aenc_detach_output 接口解除 AENC 与 ADP 的绑定关系。

```
uapi_aenc_detach_output(h_aenc, h_adp);
```

步骤 9 调用 uapi_aenc_destroy 接口销毁 AENC 模块。

```
uapi_aenc_destroy(h_aenc);
```

步骤 10 调用 uapi_adp_destroy 接口销毁数据接收端口。

```
uapi_adp_destroy(h_adp);
```

步骤 11 调用 uapi_aenc_deinit 接口去初始化 AENC 模块。

```
uapi_aenc_deinit();
```

步骤 12 调用 uapi_sea_deinit 接口去初始化 SEA 模块。

```
uapi_sea_deinit();
```

步骤 13 调用 uapi_ai_deinit 接口去初始化 AI 模块。

```
uapi_ai_deinit();
```

步骤 14 调用 uapi_adp_deinit 接口去初始化 ADP 模块。

```
uapi_adp_deinit();
```

----结束

8.3 注意事项

请严格按照“[8.2.3 语音识别场景](#)、[8.2.4 语音通话场景](#)”描述的步骤进行接口调用。

9 VAD

9.1 概述

9.2 开发指引

9.3 注意事项

9.1 概述

VAD (Voice Activity Detection) 即语音活动检测。VAD 分为模拟 VAD 和数字 VAD，模拟 VAD 是在时域进行语音能量检测，数字 VAD 是在频域进行语音能量检测。模拟 VAD 工作时，只需将模拟 MIC 和模拟 VAD 模块打开，其他电路和处理器可以下电，因此可获得极低的语音待机功耗。数字 VAD 支持数字 MIC 或 ADC 两种通路，工作时要将相应的模块上电。模拟 VAD 相比数字 VAD 可获得更低的待机功耗，但由于时域电路算法简单，存在易激活的问题。

📖 说明

VAD 当前只支持数字 VAD，需要先打开 AI 模块。

9.2 开发指引

9.2.1 接口说明

暂不提供对外接口。

9.2.2 语音活动检测场景

说明

打开语音活动检测功能，用于检测数字 MIC 是否有语音信号输入。

语音活动检测初始化步骤

语音活动检测初始化步骤如下：

步骤 1 调用 uapi_ai_init 接口初始化 AI 模块。

```
uapi_ai_init();
```

步骤 2 设置默认 AI 端口，调用 uapi_ai_get_default_attr 接口获取 AI 端口默认属性。

```
uapi_ai_port ai_port = UAPI_AI_PORT_PDM0;  
uapi_ai_attr ai_attr;  
uapi_ai_get_default_attr(ai_port, &ai_attr);
```

步骤 3 修改 AI 端口属性，调用 uapi_ai_open 接口打开音频输入设备。

```
ai_attr.pcm_attr.channels = UAPI_AUDIO_CHANNEL_1;  
ai_attr.pcm_attr.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;  
ai_attr.pcm_attr.sample_rate = UAPI_AUDIO_SAMPLE_RATE_16K;  
ai_attr.pcm_attr.sample_per_frame = ai_attr.pcm_attr.sample_rate / 100; /* 10ms */  
ai_attr.port_attr.pdm.rx_type = UAPI_AI_RX_MAD;  
ai_attr.port_attr.pdm.i2s_attr.channels = UAPI_AUDIO_CHANNEL_1;  
ai_attr.port_attr.pdm.i2s_attr.bit_depth = UAPI_AUDIO_BIT_DEPTH_16;  
td_handle h_ai = 0;  
uapi_ai_open(&h_ai, ai_port, &ai_attr);
```

步骤 4 调用 uapi_ai_register_event_proc 接口注册 AI 事件处理函数（VAD 事件等）。

```
td_void *context;  
uapi_ai_register_event_proc(h_ai, ai_event_proc, context); /* ai_event_proc 为用户自行注册的事件  
处理函数 */
```

步骤 5 调用 uapi_ai_reset_vad 接口启动一次语音活动检测。

```
uapi_ai_reset_vad(h_ai);
```

----结束

语音活动检测去初始化步骤

语音活动检测去初始化步骤如下：

步骤 1 调用 uapi_ai_close 接口关闭 AI。

```
uapi_ai_close(h_ai);
```

步骤 2 调用 uapi_ai_deinit 接口去初始化 AI 模块。

```
uapi_ai_deinit();
```

-----结束

9.3 注意事项

请严格按照“[9.2.2 语音活动检测场景](#)”描述的步骤进行接口调用。

10 VENDOR

[10.1 概述](#)

[10.2 开发指引](#)

10.1 概述

VENDOR 的作用是对接外置 DAC 或 SmartPA，提供驱动注册函数，方便对其驱动进行初始化。

10.2 开发指引

10.2.1 接口说明

VENDOR 模块实现了以下功能：外置 DAC 或 SmartPA 驱动初始化接口注册功能，该功能模块提供以下 API：

- `uapi_audio_vendor_register`：注册初始化接口。

11 ANC

 说明

当前版本暂不支持 ANC。

Draft

12 HAID

📖 说明

当前版本暂不支持 HAID。

Draft